iworkhere API Server — User and Integrator Guide

Audience: application developers, client integrators, database operators, and maintainers
Status: describes the checked-in code and documentation reviewed on 17 October 2026
API version: `v1`
Canonical API host: `https://api.iworkhere.com`

PDF edition: Download the printable user and integrator guide

> This guide describes the current implementation, including limitations that affect how clients should integrate. It is not a promise that every route is production-enabled, that every operation requires authentication, or that example application names already exist in a particular database.

Contents

1. What the API server is
2. Quick start for API consumers
3. Clients, applications, origins, and tenants
4. Database organization and schema segmentation
5. Authentication and account workflows
6. Understanding and using Swagger
7. API capabilities and endpoint map
8. Common integration examples
9. Environment configuration reference
10. Running, deploying, and operating the service
11. Troubleshooting
12. Glossary
13. Footnotes and sources
14. Alphabetical index

1. What the API server is

The iworkhere API is a Node.js and TypeScript service built with Express 5. It provides versioned HTTP endpoints for several client applications and shared platform capabilities: account registration and sign-in, OAuth and passkeys, configuration, localization, health and monitoring, Warframe data, and JoinAUnion inquiry and visit analytics.

The API is a backend, not a website framework or a database exposed directly to clients. A web, mobile, or desktop application sends HTTP requests; the API validates requests, applies shared middleware, executes application logic, accesses CockroachDB through Drizzle ORM, and may use Redis for caching. Email and translation integrations are optional external services.

```
Browser / app / service
        │ HTTPS on the public internet
        ▼
Reverse proxy (for example, nginx)
        │ HTTP on a protected internal network
        ▼
Express API :4300
   ├── tenant and request context
   ├── route validation, authentication and cache controls
   └── feature services
        ├── CockroachDB / PostgreSQL wire protocol
        ├── Redis
        └── email and optional translation providers
```

The Node server listens on internal HTTP port `4300`; the production design expects HTTPS to terminate at the reverse proxy.[1] All versioned routes use the `/v1` prefix. The route loader discovers handlers from the `src/routes/v1` directory tree and applies shared request processing.

The API does not provide a conversational AI assistant or general-purpose AI endpoint. It does contain an optional translation service that can call DeepL or Gemini when configured. AI is also documented as a development-time engineering aid; the JoinAUnion proposal mentions a user-facing AI assistant as a possible future feature, not one users can currently call.

2. Quick start for API consumers

2.1 Choose the API base URL

For the public deployment, use:

```
https://api.iworkhere.com
```

For local development, the server listens on port `4300` at the configured `HOST_IP`, for example:

```
http://localhost:4300
```

The public hostname, DNS, TLS certificate, proxy routing, and API process must all be configured by the operator. A hostname in an example is not proof that its application is registered or deployed.

2.2 Make a request

The following health check does not contain a credential:

```
curl --include https://api.iworkhere.com/v1/health/api
```

The `/v1/health/api` route is a basic starting point for checking whether the API responds. Database and memory health routes provide more specific operational signals; whether those checks are publicly exposed should be decided at the deployment edge.

2.3 Read a route in the API reference

An endpoint is the combination of an HTTP method and a path. For example:

```
POST https://api.iworkhere.com/v1/auth/login
```

Use the method and path shown in the API reference. Send JSON using `Content-Type: application/json` when the operation accepts a JSON body. Read the endpoint's required fields, validation rules, response, and authentication requirements before integrating it.

2.4 Keep credentials on the server side

Never put database passwords, API-provider keys, OAuth client secrets, SMTP credentials, token-signing secrets, private TLS keys, or user access tokens in a public web bundle, source-controlled example, URL, or diagnostic log. Store client-specific secrets in a server-side secret manager. A browser app may send an access token to the API in an HTTPS authorization header, but must not embed server credentials.

3. Clients, applications, origins, and tenants

These concepts have related names but do different jobs. Treating them as interchangeable leads to incorrect routing and unsafe assumptions.

Concept	What it identifies	How the current service uses it
Client	The web, mobile, or other software making an API request	Supplies request data and, for browser flows, an `Origin` header
Application	A registered API consumer, represented by a row in `public.applications`	`app_key` resolves an enabled application; issued tokens are associated with its application ID
Application origin	An exact web origin approved for an application	An enabled row in `public.application_origins` can bind an `Origin` or `Referer` to an enabled application
Tenant	A database-schema selection for a request	Tenant middleware selects a schema name and the database session searches that schema before `public`
CORS origin	A browser permission to make/read a cross-origin request	Controlled by built-in origin rules and `CORS_ALLOWED_ORIGINS`; it does not register an application or authenticate a user

3.1 Application registration is not DNS

An application key such as `bill.workhere.com` or `joinaunion.iworkhere.com` is an identifier stored in `public.applications`; it is not a secret. Authentication resolves that key to an enabled application record. Browser-based application resolution can instead use an exact enabled origin registered in `public.application_origins`.[2]

Consequently, pointing a domain at the API is not enough to enable sign-in. The operator must configure the application record and, for browser sign-in, its permitted origin record. Registration, OAuth, passkeys, and other application-aware operations must use the key/origin belonging to the intended consuming application.

3.2 How a tenant is selected

The current global tenant middleware chooses a tenant using this order:

1. `X-Tenant` request header, if present; the value before the first `.` is used.
2. The first hostname label found in `Origin`, `Referer`, or the supported origin-related headers.
3. The request hostname as a fallback.
4. For generic API or local hostnames, `app_key` in the query or JSON body as a fallback.
5. If the resolved value remains generic, it defaults to `localhost`. Special `accounts` and `google` labels are mapped to `public`.

For example, an `Origin: https://joinaunion.iworkhere.com` ordinarily selects tenant name `joinaunion`; `X-Tenant: bill` selects `bill`. A value such as `X-Tenant: bill.workhere.com` is reduced to `bill`.

`X-Tenant` is a routing hint, not proof of identity or authorization. The current resolver does not consult the application registry to authorize tenant choices. Do not let an untrusted end user choose arbitrary tenant headers. A trusted client backend or reverse proxy should control this value, and deployment

operators must ensure tenant names map to valid, intended database schemas.[3]

The current tenant middleware interpolates the selected tenant into a SQL `SET search_path` statement and does not validate it against a schema allowlist. Only trusted infrastructure should be able to influence tenant selection; production deployments should validate schema identifiers against an allowlist before using this mechanism.

3.3 Examples: Bill and JoinAUnion clients

Consumer	Example application key	Example browser origin	Expected schema tenant from that origin
Bill client	`bill.workhere.com`	`https://bill.workhere.com`	`bill`
JoinAUnion client	`joinaunion.iworkhere.com`	`https://joinaunion.iworkhere.com`	`joinaunion`

These are illustrative identities. Confirm that the exact application key is enabled and that the exact origin is registered before relying on it. An application key should be sent in the endpoint's documented body or query parameter; it does not replace `Authorization: Bearer ...` on protected endpoints.

Browser CORS is separate. The API allows HTTPS origins matching a single subdomain of `iworkhere.com` by default, plus exact origins in `CORS_ALLOWED_ORIGINS`. Thus `https://joinaunion.iworkhere.com` matches the built-in pattern, while `https://bill.workhere.com` needs an explicit CORS allowlist entry. CORS acceptance does not by itself make an origin valid in `application_origins`; that registry check is separate.[4] Non-browser clients such as `curl` are not restricted by browser CORS enforcement.

4. Database organization and schema segmentation

4.1 One database, multiple schemas

The service connects to a PostgreSQL-compatible database (CockroachDB in the documented deployment) and uses database schemas as namespaces. A tenant schema is not a separate database server or database account. Schema names include `bill`, `joinaunion`, and `michael`; `public` contains shared tables.[5]

The request middleware acquires a dedicated connection from the PostgreSQL pool, executes:

```
SET search_path TO <tenant>, public;
```

It constructs a Drizzle client bound to that connection and stores it in `AsyncLocalStorage`. Calls through the shared `db` proxy use the request-scoped client when available. At response completion or connection close, the middleware resets `search_path` and returns the connection to the pool.[6]

The middleware is named `tenantTransaction`, but its current implementation does not issue `BEGIN`, `COMMIT`, or `ROLLBACK` around the request. Do not assume that all database changes in a request are atomic as a single transaction. Individual operations may use explicit Drizzle transactions where implemented.

4.2 What `search_path` means

For an unqualified table name, the database looks through schemas in `search_path` order. With a request tenant of `joinaunion`, an unqualified query for `config` resolves to `joinaunion.config` if that table exists there; otherwise it can resolve to `public.config`.

```
-- With search_path set to joinaunion, public:
SELECT * FROM config;

-- This always names the JoinAUnion table, regardless of search_path:
SELECT * FROM joinaunion.visit_info;

-- This always names the shared/public table:
SELECT * FROM public.applications;
```

In Drizzle definitions, `pgTable( ... )` declares an unqualified table name and therefore follows the active search path. By contrast, the JoinAUnion visit, device, and widget tables are declared with `pgSchema('joinaunion')`; they are schema-qualified and always point to the JoinAUnion schema.[7]

4.3 Shared tables versus tenant tables

The intended placement documented by the schema and migrations is:

Placement	Representative tables	Meaning
`public` shared identity/application data	`users`, `user_statuses`, `applications`, `application_origins`, `user_applications`, `auth_tokens`, password and email verification records, `passkey_credentials`	Shared identities, application registry, and credentials. Application association is represented by foreign keys such as `application_id`; it is not a separate copy of these tables for every client.
`public` shared/reference data	`config`, `localizations`, `countries`, and other unqualified platform tables	Shared configuration or reference data, subject to the active search path and actual deployed schemas.
`joinaunion` explicitly qualified data	`joinaunion.visit_info`, `joinaunion.user_devices`, `joinaunion.widget_answers`	JoinAUnion visit events, devices, and inquiry submissions. These table definitions do not switch schemas when another tenant is selected.
Tenant-resolved product tables	`warframes`, `warframe_weapons`, `warframe_modules`	Unqualified Drizzle tables that resolve using the tenant-first search path. Migration `0022` moves the existing Warframe tables to `michael`.

The original migration creates tenant schemas `bill`, `joinaunion`, and `michael` and moves the Warframe tables to `michael`.[8] Later schema creation or migration changes must be checked against the deployed database; do not infer current table placement from a hostname alone.

Important shadowing rule: if a tenant schema contains a table with the same unqualified name as a `public` table, the tenant table wins because it appears earlier in the search path. Conversely, `public.table_name` and `joinaunion.table_name` explicitly bypass the search-path choice. When diagnosing a query, record the request tenant, the table's Drizzle declaration (`pgTable` or `pgSchema`), and the actual database schema.

4.4 Safe database access expectations

- Use the request-scoped `db` proxy for tenant-aware ORM work.
- Do not assume the exported raw `query()` helper is tenant-scoped: it calls the shared pool directly.
- Do not build SQL by concatenating tenant names or user input.
- Use explicit schema qualification where the data's ownership is intentionally fixed.
- Apply schema changes through reviewed Drizzle migrations; back up and verify production data before applying them.
- Do not create a table in `public` just to make an endpoint work until its data ownership and cross-tenant consequences are understood.

5. Authentication and account workflows

5.1 Local registration and login

Registration is application-scoped. A typical registration request contains the application key, username, email, and password. Passwords are hashed by the service; registration creates a pending account and sends a time-limited email verification code when email delivery is configured.

Example login request (replace the example host, username, and password with authorized test credentials; never commit real credentials):

```
curl --request POST 'https://api.iworkhere.com/v1/auth/login' \
  --header 'Content-Type: application/json' \
  --data '{
    "app_key": "joinaunion.iworkhere.com",
    "identifier": "YOUR_TEST_USERNAME_OR_EMAIL",
    "password": "YOUR_TEST_PASSWORD"
  }'
```

A successful login returns user/application metadata and an access/refresh token pair. Treat both token values as secrets. Access-token lifetime and refresh-token lifetime are controlled by the corresponding environment settings described in Chapter 9.

5.2 Use an access token

For endpoints whose route module requires authentication, send the access token using the Bearer scheme:

```
curl 'https://api.iworkhere.com/v1/auth/me' \
  --header 'Authorization: Bearer YOUR_ACCESS_TOKEN'
```

The server hashes the presented token and validates token type, expiry, and revocation. Redis may cache valid token lookups until expiry. Refresh operations rotate tokens: clients should replace the old pair with the returned pair and must not expect a refresh token to be reusable indefinitely.

5.3 Other sign-in and recovery options

- Google OAuth 2.0: uses application context and signed OAuth state; the provider client and redirect URLs must be configured.
- Passkeys / WebAuthn: the client completes registration or login options/verification ceremonies; valid origins and relying-party identity must be registered/configured. See WebAuthn deployment notes.
- Email verification: registration verification and resend operations use application context and expiring codes.
- Password recovery: initiate, verify, and complete the flow using the documented reset endpoints. Completion updates password history and revokes existing auth tokens.
- Token revocation: use the token endpoint to revoke the current token/session according to the API contract.

5.4 Authentication is route-specific

Do not infer authentication requirements from the URL prefix, the Swagger UI, or an endpoint's prose alone. Runtime route registration reads the route module's `authRequired` export. If the module does not set it to `true`, the route loader treats that method as not requiring bearer authentication. The checked-in Swagger generation currently does not emit an OpenAPI security scheme or per-operation security requirements.[9]

In particular, the current JoinAUnion visit-management `GET` and `POST` routes and synthetic-data `POST /generate` export `authRequired = false`. Generation deletes all existing `joinaunion.visit_info` rows with the supplied `location_source` before inserting generated data. Production adds a `confirm_delete` confirmation requirement for that destructive operation, but this is not authentication or authorization. Restrict these endpoints at a trusted deployment boundary or change their runtime access controls before exposing them to untrusted callers.

6. Understanding and using Swagger

6.1 Where to find it

In non-production environments, the server mounts Swagger UI at:

```
http://<HOST_IP>:4300/docs
```

The server intentionally does not mount the UI when `NODE_ENV=production`. The generated OpenAPI JSON snapshot is checked into `swagger.json`. The project command `npm run swagger:gen` regenerates it from public route docblocks.[10]

6.2 Read an operation

In Swagger UI:

1. Expand a tag/category and choose an operation.
2. Check the HTTP method and complete path.
3. Read the summary and description; note special behavior and side effects.
4. Review query parameters and request examples.
5. Select Try it out, enter non-sensitive test inputs, and execute against an authorized non-production server.
6. Inspect the response status, headers, and JSON body.

Swagger is useful for discovery and manual experimentation. It does not create an account, grant access, supply secrets, or guarantee that the current deployment is configured for the operation. Use only test identities and approved environments.

6.3 What the current OpenAPI document does and does not describe

The project generator scans route TypeScript files, includes handlers marked `@external` in their first `@myDocBlock`, and writes path, method, summary, description, query parameters, and generic success/error response descriptions. It does not currently generate a full OpenAPI security model or comprehensively model JSON request bodies and response schemas. Some docblocks or checked-in paths can drift from runtime route placement. For example, runtime routes are discovered from their filesystem location under `src/routes/v1`, while Swagger may use an explicit `@path` annotation.

If Swagger and runtime disagree, verify the deployed route and method against the current route module and the runtime route loader before shipping an integration. The actual runtime route path is derived from the route's directory under `src/routes/<API_VERSION>`, not from Swagger alone.

7. API capabilities and endpoint map

Paths below are the runtime route structure for the current `API_VERSION=v1` source tree. Authentication is per method and must be verified in Swagger plus the current route module.

Capability	Runtime route examples	Purpose
Account lifecycle	`/v1/auth/register`, `/v1/auth/login`, `/v1/auth/me`, `/v1/auth/upgrade`	Create, authenticate, inspect, or upgrade a local account
Tokens	`/v1/auth/refresh`, `/v1/auth/token`	Rotate or revoke auth tokens
Google OAuth	`/v1/auth/oauth/google`, `/v1/auth/oauth/google/callback`	Start and complete Google OAuth
Email and password recovery	`/v1/auth/emailverify`, `/v1/auth/emailverify/resend`, `/v1/auth/passreset/*`	Verify email, resend a code, and recover an account
Passkeys	`/v1/auth/passkeys`, `/v1/auth/passkeys/register/*`, `/v1/auth/passkeys/login/*`	Enroll, authenticate with, list, and revoke passkeys
Device management	`/v1/auth/devices`	Read/manage device records through the available methods
JoinAUnion contact widget	`/v1/clients/joinaunion/widget`	Validate, persist, and email an inquiry
JoinAUnion visit analytics	`/v1/clients/joinaunion/visit_info/config`, `/manage`	Discover filter values and query visit/journey data
JoinAUnion synthetic visits	`/v1/clients/joinaunion/visit_info/generate`	Replace a labelled dataset with generated analytics fixtures
Warframe domain data	`/v1/clients/warframe/modules`, `/warframes`, `/weapons`	Read/write product data using the request tenant's search path
Configuration and localization	`/v1/config`, `/v1/localization`	Access or update shared configuration and language content
Health and monitoring	`/v1/health/*`, `/v1/monitor/*`	Inspect service, database, memory, network, localization, and visit signals
Generated README content	`/v1/readme`	Retrieve the repository README rendered as HTML

An endpoint may return `400` for invalid input, `401` for missing/invalid authentication where required, `403` for disabled application/origin cases, `404` for an unknown route/resource, `405` for an unsupported method, or `500` for an unexpected server error. Use each operation's documented response and avoid treating these examples as an exhaustive error contract.

8. Common integration examples

8.1 Browser application sign-in

For a browser application at `https://joinaunion.iworkhere.com`:

1. Ask the operator to confirm that the application exists and is enabled in `public.applications`.
2. Ask the operator to register that exact HTTPS origin in `public.application_origins`.
3. Confirm that the browser origin passes CORS. The current built-in rule accepts a single HTTPS subdomain of `iworkhere.com`.
4. Call login with the corresponding `app_key` (or use the API's documented origin-based flow where supported).
5. Keep the returned access token in the client application's session handling; send it as a Bearer token only to protected API routes.

Do not set `X-Tenant` in browser code based solely on an untrusted URL or a user-controlled value. Tenant selection is a server routing concern, not a substitute for checking a user's permissions.

8.2 Server-to-server request for a tenant-specific data path

For an authorized backend that has been explicitly assigned to the `bill` tenant:

```
curl 'https://api.iworkhere.com/v1/clients/warframe/warframes' \
  --header 'Accept: application/json' \
  --header 'X-Tenant: bill' \
  --header 'Authorization: Bearer YOUR_ACCESS_TOKEN'
```

This example illustrates header syntax only. Confirm the actual route, its method, and its auth policy against the running API. The current route tree for Warframe handlers includes `/clients/warframe/...`; some generated Swagger annotations use a different path. `X-Tenant` selects the unqualified database schema; it does not verify that the token belongs to the intended application or tenant.

8.3 JoinAUnion inquiry

The widget accepts an inquiry body with recipient, sender email, message, province, and industry:

```
curl --request PUT 'https://api.iworkhere.com/v1/clients/joinaunion/widget' \
  --header 'Content-Type: application/json' \
  --header 'Origin: https://joinaunion.iworkhere.com' \
  --data '{
    "recipient": "organizer@example.invalid",
    "email": "worker@example.invalid",
    "message": "Example inquiry for a test environment.",
    "province": "Ontario",
    "industry": "Healthcare"
  }'
```

This example uses reserved `.invalid` addresses and non-personal text. In a real deployment, the recipient must be appropriate, email delivery must be configured, and data-handling/consent language should match the product. Do not submit real personal information to a test system.

8.4 Explore API documentation without exposing credentials

Use the non-production Swagger UI to inspect schemas and try operations with synthetic data. If an operation requires a bearer token, obtain a temporary token for a dedicated test account, use the UI only over a trusted connection, and revoke the token afterward. Do not paste production credentials into a public Swagger instance.

9. Environment configuration reference

9.1 Secret-handling guarantee

This inventory lists names and purposes only. It deliberately does not reproduce, summarize, validate, or disclose any value from `.env.development`. The source file is ignored by Git; keep it that way. The table is based on its variable names and source-code references, not on the value currently configured on any machine.

There are 46 distinct variable names in the local `.env.development` file reviewed for this guide. "Sensitive" means the value should be treated as secret or operationally private. "Not read by current source" means no current source-code reference was found during this review; it may be legacy or used by deployment tooling outside this repository. Do not delete a local setting solely on that basis without confirming with the operator.

Application and runtime

Variable	Purpose in this project	Sensitive?	Current-source note
`NODE_ENV`	Selects the environment; affects production behavior, cookies, and whether Swagger UI is mounted.	No	Read
`HOST_IP`	Interface/address passed to the HTTP server bind call.	Operational	Required by server startup
`API_VERSION`	Selects the route directory and version prefix, currently `v1`.	No	Read
`APP_KEY`	Fallback application identity used by the admin UI when its form does not supply one.	No	Read by admin route
`APP_URL`	Fallback client/application URL for origin resolution and selected OAuth/email flows.	No	Read
`MAX_CONCURRENT_REQUESTS`	Global in-process concurrency throttle for loaded route handlers.	No	Read
`DEBUG`	Enables selected diagnostic logging and local development CORS behavior.	No	Read
`AUTH_ME_DEBUG`	Enables detailed `/auth/me` and app-level JSON-request diagnostics. May expose request metadata in logs.	Operational	Read; avoid in production
`AUTH_MW_DEBUG`	Enables authentication middleware diagnostics.	Operational	Read; avoid in production
`AUTH_OA_DEBUG`	Appears to be an OAuth/auth diagnostic setting.	Operational	Not read by current source

Database, certificates, and cache

Variable	Purpose in this project	Sensitive?	Current-source note
`DB_HOSTNAME`	Database server host and TLS server name.	Operational	Required

Variable	Purpose in this project	Sensitive?	Current-source note
DB_PORT	Database server port.	Operational	Required by app; migration config has a fallback
DB_NAME	Database name.	Operational	Required
DB_USER	Database login name; also used in client-certificate filenames.	Yes / operational	Required
DB_CERT_DIR	Directory containing the CA and database client TLS certificate/key files.	Operational	Required
DB_HOST	Alternate/older database-host setting.	Operational	Not read by current source; app uses DB_HOSTNAME
API_CERT_DIR	API certificate-directory setting.	Operational	Not read by current source; the Node server itself uses internal HTTP
ROOT_CA_CERT	Root certificate path/name setting.	Operational	Not read by current source
REDIS_SERVER_PASSWORD	Password for the Redis client.	Yes	Read

The Redis client also recognizes REDIS_HOST and REDIS_PORT, defaulting in code when they are absent; these two names are not defined in the reviewed .env.development inventory.

Authentication, identity providers, and tokens

Variable	Purpose in this project	Sensitive?	Current-source note
ACCESS_TOKEN_TTL_SECONDS	Access-token lifetime, in seconds.	No	Read; minimum is 60 seconds
REFRESH_TOKEN_TTL_SECONDS	Refresh-token lifetime, in seconds.	No	Read; minimum is 300 seconds
EMAIL_VERIFY_TOKEN_TTL_SECONDS	Email verification-token lifetime, in seconds.	No	Read
RESET_TOKEN_TTL_SECONDS	Password-reset-token lifetime, in seconds.	No	Read; must be positive
OAUTH_STATE_SECRET	HMAC signing secret for OAuth state.	Yes	Required when OAuth configuration is used
GOOGLE_OAUTH_CLIENT_ID	OAuth client identifier.	No	Read
GOOGLE_OAUTH_CLIENT_SECRET	OAuth client secret.	Yes	Read
GOOGLE_AUTHORIZATION_URL	Google OAuth authorization endpoint.	No	Read
GOOGLE_TOKEN_URL	Google OAuth token endpoint.	No	Read

Variable	Purpose in this project	Sensitive ?	Current-source note
GOOGLE_USERINFO_URL	Google user-info endpoint.	No	Read
GOOGLE_OAUTH_REDIRECT_URI	OAuth callback URL registered with the provider.	No	Read
GOOGLE_OAUTH_SUCCESS_REDIRECT_URL	Destination after successful OAuth.	No	Read
GOOGLE_OAUTH_FAILURE_REDIRECT_URL	Destination after failed OAuth.	No	Read
GOOGLE_OAUTH_WEB_CLIENT_CREATION_DATE	Metadata associated with a Google web client.	No	Not read by current source
GOOGLE_OAUTH_WEB_JSON	Google web client configuration blob. It may contain sensitive fields depending on its contents.	Treat as sensitive	Not read by current source
WEBAUTHN_CEREMONY_SECRET	Signs short-lived passkey ceremony data.	Yes	Required for passkey ceremony operations
SEED_BILL_PASSWORD	Password consumed by the Bill default-login seed utility.	Yes	Seed-only; never use as an API integration credential

Browser access, email, and optional translation

Variable	Purpose in this project	Sensitive?	Current-source note
CORS_ALLOWED_ORIGINS	Comma-separated exact browser origins added to the CORS allowlist.	No	Read; enter origins, not paths
SMTP_HOST	Outbound mail server host.	Operational	Read
SMTP_PORT	SMTP port; the current mailer enables secure SMTP when the port is 465.	Operational	Read
SMTP_USER	SMTP authentication username.	Yes / operational	Read
SMTP_PASS	SMTP authentication password.	Yes	Read
SMTP_FROM_EMAIL	Sender address used by the mailer.	Operational	Read
SMTP_KEY	SMTP/provider key setting.	Yes	Not read by current mailer source
BREVO_KEY	Brevo provider key setting.	Yes	Not read by current source; current mail delivery is configured via SMTP
DEEPL_API_KEY	DeepL translation API credential.	Yes	Read by optional translation service
GEMINI_KEY	Gemini translation API credential.	Yes	Read by optional translation service

9.2 Additional runtime settings not present in `.env.development`

The following names are recognized by code or configuration helpers but are not in the 46-name inventory above:

Variable	Purpose
`CLEANUP_JOB_INTERVAL_MS`	Background expired-token cleanup interval; the server defaults to one hour when unset.
`ROUTE_LOADER_DEBUG`	Enables detailed route-loader diagnostics when set to `1`.
`ALLOW_MASS_DELETE`	Can satisfy the generator's production mass-delete confirmation check when set to `true`; it does not authenticate the caller.
`REDIS_HOST`, `REDIS_PORT`	Optional Redis host/port; code defaults to loopback and the standard Redis port.
`WEBAUTHN_RP_NAME`, `WEBAUTHN_CEREMONY_TTL_SECONDS`, `WEBAUTHN_PREVIOUS_CEREMONY_SECRET`	Optional WebAuthn display name, ceremony lifetime, and temporary prior signing key during rotation.
`WEBAUTHN_RP_ID`	Optional compatibility setting for direct service callers; HTTP passkey routes resolve relying-party identity from registered application origins.
`DEEPL_KEY`, `DEEPL_AUTH_KEY`, `GEMINI_API_KEY`	Translation-service aliases accepted by source code.

The code also uses fixed/default runtime behavior not controlled by `.env.development`: the HTTP listener uses port `4300` (it does not read `PORT`), the Redis client defaults to localhost, and production disables Swagger UI.

9.3 Configuration precedence

The shared configuration helper intends to load `.env`, `.env.local`, `.env.<NODE_ENV>`, and `.env.<NODE_ENV>.local`, then let process environment values take precedence. There is a precedence caveat in the current implementation: each file is loaded with dotenv's default non-overriding behavior, and values from the first file loaded are added to `process.env`. Those values can then override later file values during the final merge. For a given key, an initially supplied process environment value wins; otherwise, the first loaded file defining that key can win over later files. In test mode `.env.development` is appended as a fallback. Configuration files are resolved relative to the process working directory, so start commands must run from the repository root. Until precedence is corrected, avoid defining the same key in multiple `.env*` files and use deliberate process environment overrides.

Keep actual values out of this guide, screenshots, issue reports, and commits. If a secret has been exposed, treat it as compromised and rotate it; deleting the visible text alone does not invalidate the credential.

10. Running, deploying, and operating the service

10.1 Local development

Prerequisites: Node.js, npm, access to a compatible CockroachDB/PostgreSQL service, database TLS certificates, and Redis. Email, OAuth, and translation providers are needed only when testing those integrations.

```
npm ci
npm run build
npm test
npm run dev
```

The `dev` script starts the TypeScript server. The API listens on port `4300`; local Swagger UI is available at `/docs` when the server is not in production mode. Do not publish local database credentials or certificate files.

10.2 Database schema changes

Schema definitions are in `src/db/schema`; SQL migration files are in `src/db/migrations`. Typical project commands:

```
npm run drizzle:check
npm run drizzle:generate
npm run drizzle:apply
```

Review generated SQL and the target schema before applying a migration, especially when tables are explicitly schema-qualified or have tenant-specific ownership. For production, schedule a backup, apply migrations using an operator-controlled identity, and verify the resulting schema and indexes.

10.3 Production boundary

- Terminate public TLS at a properly configured reverse proxy; bind the Node listener only to an intended internal interface.
- Keep database, Redis, email, OAuth, WebAuthn, and translation credentials in a secret manager or protected process environment.
- Configure CORS and the application-origin registry deliberately; one does not replace the other.
- Restrict diagnostic endpoints and debug settings.
- Protect unauthenticated or destructive routes at the application and deployment layers.
- Treat API logs as sensitive. The current tenant middleware logs request headers, which can include `Authorization` and cookies; restrict log access and retention, and add header redaction before using this code in an environment where clients send credentials. `AUTH_ME_DEBUG` can also log full JSON request bodies. Do not enable verbose request diagnostics in production.
- Apply the principle of least privilege to database users and ensure schema privileges match the intended tenant model.
- Monitor service logs, health routes, database connectivity, Redis, mail delivery, and backup/restore readiness.

The checked-in CI workflow currently runs the TypeScript build and a critical-level npm audit; the full Vitest suite and ESLint are separate local commands.

11. Troubleshooting

Symptom	Checks
`401 Unauthorized`	Confirm the route requires bearer auth, send an access token rather than a refresh token, and verify it has not expired or been revoked.
Application not found/disabled	Verify `app_key` exactly matches an enabled application record; for origin-based resolution, verify the exact origin and both enabled flags.
Browser CORS error	Check the browser's exact `Origin`, the built-in HTTPS subdomain rule, and `CORS_ALLOWED_ORIGINS`. Also check `application_origins` separately for auth flows.
Data appears under the wrong schema	Inspect `X-Tenant`, `Origin`/`Referer`, `Host`, and `app_key` resolution; inspect `search_path`; check whether the Drizzle table uses `pgTable` or explicit `pgSchema`.
An explicitly qualified table ignores tenant choice	Expected for `joinaunion.*`: explicit schema qualification does not follow the request search path.
Database connection fails during startup	Verify database host/name/user/port, TLS certificate directory and required certificate filenames, server trust, and network reachability. Do not print private key contents when reporting the problem.
Redis errors or passkey replay/cache problems	Verify the Redis endpoint and credentials in the service environment; check the configured Redis database and service connectivity.
Swagger `/docs` returns not found	It is mounted only when `NODE_ENV` is not `production`; consult `swagger.json` or use a non-production deployment.
Swagger path differs from the running route	Compare the route directory and `API_VERSION` with the docblock `@path`; runtime route discovery determines the actual registered path.
Email is not delivered	Check SMTP connectivity, sender configuration, and provider acceptance; inspect application logs and email-audit records using authorized operator access.
Synthetic visit generation changes data	The endpoint replaces rows matching the submitted location-source label. Use an isolated test label and dataset; never run it against real data without explicit operational approval.

When reporting a bug, include the method, path, timestamp/time zone, sanitized request shape, response status, request ID if available, and the relevant tenant/application identifiers. Remove bearer tokens, passwords, email contents, database details, personal data, and all environment values.

12. Glossary

Term	Definition
API	HTTP interface through which client software uses server-side capabilities.
Application key (`app_key`)	Public identifier for a registered consuming application; not a password or access token.
Application origin	Exact browser origin registered to an application for origin-based authentication flows.
Bearer token	Secret access credential sent in the `Authorization` header. Anyone who obtains it may be able to act as its user until it expires or is revoked.
CORS	Browser-enforced cross-origin resource sharing policy. It is not user authentication.
Drizzle ORM	TypeScript database mapping/query layer used by the API.
NDJSON	Newline-delimited JSON; a streaming representation with one JSON value per line.
OpenAPI / Swagger	Machine-readable API description format and interactive browser UI for exploring operations.
PostgreSQL `search_path`	Ordered schema lookup path used to resolve unqualified table names.
Schema	A namespace containing database objects such as tables; this project uses schemas for selected tenant/product data.
Tenant	Current request's schema-selection label; not inherently an authenticated user or an application identity.
TLS	Transport encryption and peer authentication used for public HTTPS and the database connection.

13. Footnotes and sources

Footnotes

Further reading

- Project overview and executive documents
- WebAuthn deployment configuration
- Public/shared data inventory and migration notes
- Authentication edge-case test coverage
- Email-verification workflow notes
- Contribution guide
- OWASP API Security Project
- OpenAPI Specification
- CockroachDB schema documentation
- PostgreSQL schema search path documentation

14. Alphabetical index

Entry	See
Access token	Authentication
Application key (`app_key`)	Clients and applications
Application origin	Clients and applications
CORS	Clients and applications, Configuration
Database migration	Operations
Environment variables	Configuration reference
JoinAUnion	Endpoint map
OpenAPI / Swagger	Swagger guide
`public.table_name`	Database schemas
`joinaunion.table_name`	Database schemas
Request examples	Common integrations
`search_path`	Database schemas
Tenant	Clients and tenants, Database schemas
WebAuthn / passkeys	Authentication, WebAuthn deployment

1. `src/server.ts` binds the HTTP server on port `4300`; its comments and startup logging describe TLS termination at the reverse proxy. Swagger UI is mounted only outside production. ↩
2. `src/services/auth/authContext.ts` resolves enabled application records. `src/services/auth/applicationOriginResolver.ts` resolves browser origins against enabled application/origin records. ↩
3. `src/middleware/tenantMiddleware.ts` implements tenant selection. `src/middleware/tenantTransaction.ts` applies the resulting name to the database session. The documentation intentionally treats tenant headers as trusted routing input, not authorization. ↩
4. `src/appFactory.ts` implements browser CORS rules; application-origin authorization is implemented separately in the auth origin resolver. ↩
5. The schemas and migration files are maintained in `src/db/schema` and `src/db/migrations`. ↩
6. `src/services/dbService.ts` exposes the `AsyncLocalStorage`-aware `db` proxy. `src/middleware/tenantTransaction.ts` scopes and releases a pooled connection. ↩
7. `src/db/schema/visit_info.ts`, `user_devices.ts`, and `widget_answers.ts` declare the JoinAUnion schema explicitly. Most shared/application tables use unqualified `pgTable` declarations. ↩

8. Migration `0022_multi_tenant_schema_setup.sql` creates the initial tenant schemas and moves Warframe tables into `michael`. ↩

9. `src/loaders/routeLoader.ts` reads each route module's `authRequired` export. The current `src/helpers/swaggerGen.ts` emits public route metadata but no OpenAPI security scheme. ↩

10. `src/loaders/swagger.ts` serves `swagger.json` through Swagger UI. `src/helpers/swaggerGen.ts` generates the checked-in document from external route annotations. ↩